SentryOps

Autonomous Multi-Agent Production Incident Response & Root Cause Analysis Platform

OPERATIONAL DOMAIN

- **Site Reliability Engineering (SRE)**: Autonomous incident triage and dynamic mitigation.
- **Unified Observability**: Tri-modal correlation across metrics, logs, and traces.
- **Enterprise Safety**: Rigorous human-in-the-loop approvals before infrastructure change.

CORE ARCHITECTURE

- **Agent Orchestration**: Python 3.11+ with LangGraph state graph coordination.
- **Frontend & Storage**: React 19, TypeScript, PostgreSQL 15 + pgvector for RAG.
- **Infrastructure**: Kubernetes cluster (Kind) with Prometheus, Loki, Jaeger & Grafana.

GOVERNING PRINCIPLE

- **Empowered AI Investigation**: Parallel investigative reasoning across telemetry stores.
- **Strict Deterministic Guardrails**: Hardcoded safety rules, zero direct cluster actuation.
- **Anti-Hallucination Gate**: Evidence IDs verified against ground truth before RCA generation.

The Modern Production Incident Dilemma

Operational Bottlenecks in Modern Cloud-Native Microservices

FRAGMENTED TELEMETRY

- **Alert Storms & Noise**: High-severity incidents trigger cascading alerts across dozens of decoupled microservices.
- **Context Switching**: On-call engineers waste critical minutes manually correlating logs across Loki, metrics across Prometheus, and traces across Jaeger.
- **Siloed Insights**: Lack of unified causality mapping delays diagnosis.

PROLONGED MTTR

- **80%+ Diagnosis Overhead**: In complex distributed architectures, identifying the root cause consumes the majority of incident recovery time.
- **Runbook Retrieval Latency**: Relevant operational runbooks and historical postmortems are rarely surfaced when outages strike.
- **Cognitive Burnout**: High-stress manual triage increases operator mistakes under severe SLA pressure.

UNCONSTRAINED AI RISKS

- **Hallucinated Commands**: Giving an LLM direct shell access or raw kubectl execution creates catastrophic reliability risks.
- **Cascading Outages**: Unbounded autonomous remediation can accidentally delete databases or terminate healthy services.
- **Missing Auditability**: Autonomous black-box actions violate compliance, governance, and organizational change controls.

The SentryOps Solution

End-to-End Autonomous Incident Lifecycle with Deterministic Safety Guardrails

- **Simulated Production Architecture**:

– 5 FastAPI microservices (`order`, `payment`, `notification`, `user`, `api-gateway`).

– Dynamic fault-injection API running inside the Kubernetes cluster on port 8005.

- **End-to-End Observability Pipeline**:

– Full OpenTelemetry instrumentation across all routes.

– Real-time Prometheus metrics, Loki structured log streaming, and Jaeger distributed traces visualized in Grafana.

- **Parallel Multi-Agent Orchestration**:

– LangGraph manages specialized agents with structured fan-in synthesis.

- **Deterministic Safety Guardrails**:

– **Strict Allowlist**: Remediations limited exclusively to `RESTART_SERVICE`, `SCALE_SERVICE`, and `ROLLBACK_SERVICE`.

– **Non-LLM Risk Engine**: Deterministic rules classify actions (`LOW`, `MEDIUM`, `HIGH`) independently of LLM reasoning.

– **Mandatory Human Approval Gate**: All medium and high-risk proposals halt at `PENDING_APPROVAL`.

– **Zero Cluster Actuation**: Agents operate against `SIMULATED_INFRA_STATE` without direct `kubectl` privilege.

- **Automated Verification & Postmortem**:

– Post-remediation health checks verify recovery and compile structured postmortems.

Multi-Agent Parallel Investigation

LangGraph Fan-Out / Fan-In Investigative Workflow & Anti-Hallucination Gates

- **Incident Trigger & Dispatch**:

– Outage detected via error-rate threshold or synthetic fault API.

– `IncidentManager` spawns parallel investigation across 6 specialized agents.

- **Parallel Specialized Agents (Fan-Out)**:

– **LogAgent**: Queries Loki for error stacks, 5xx status codes, and exceptions.

– **MetricAgent**: Queries Prometheus for error rate spikes and P95 latency.

– **TraceAgent**: Queries Jaeger for error spans, tags, and trace waterfalls.

– **DeploymentAgent**: Checks recent config changes, releases, and fault flags.

– **InfraAgent**: Scrapes node memory, CPU, and pod restart health.

– **KnowledgeAgent**: Retrieves relevant historical runbooks via pgvector RAG.

- **Structured Synthesis (Fan-In)**:

– Specialized agents populate a centralized incident evidence store.

- **RCA Agent & Anti-Hallucination Gate**:

– Performs evidence linkage and cross-source causal synthesis.

– **Strict Validation Gate**: The RCA Agent algorithmically validates that all cited evidence IDs exist in the evidence store before conclusions are finalized.

- **RemediationAgent & Risk Policy**:

– Formulates structured mitigation action matching the RCA hypothesis.

– Enforces deterministic risk assessment before any action can proceed.

- **Input-to-Output Flow Summary**:

• Telemetry & Signals (Input) ➔ Parallel Investigation ➔ Risk Gate & Verification (Output).

Deterministic Risk Policy & Approval Gate

Human-in-the-Loop Operational Safety Classification

LOW RISK POLICY

- **Policy Rule**: Restarting a single unhealthy microservice instance.
- **Autonomous Execution**: Allowed (Simulated).
- **Operator Action**: None required (automatically advances to post-remediation verification).

MEDIUM RISK POLICY

- **Policy Rule**: Scaling service replica counts (`SCALE_SERVICE`).
- **Autonomous Execution**: **Prohibited**.
- **Operator Action**: **Mandatory Operator Approval** required via dashboard before execution.

HIGH RISK POLICY

- **Policy Rule**: Multi-instance restarts or configuration rollbacks (`ROLLBACK_SERVICE`).
- **Autonomous Execution**: **Prohibited**.
- **Operator Action**: **Mandatory Operator Approval** required.

- **State Machine Enforcement**:
  - Any proposal classified as `MEDIUM` or `HIGH` automatically transitions to `PENDING_APPROVAL` and halts execution.
- **Operator Approval API (`POST /incidents/{id}/approve`)**:
  - Records approval audit trail, transitions status to `APPROVED`, and triggers deterministic simulated execution.
- **Operator Rejection API (`POST /incidents/{id}/reject`)**:
  - Transitions status to `REJECTED`, cancels remediation execution, and returns incident to `INVESTIGATING`.

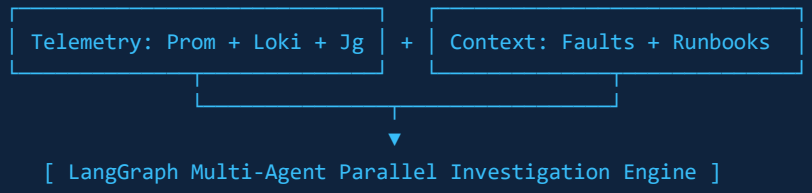
Project Input: What Goes Into SentryOps?

Production Telemetry, Incident Signals, System Context & Operational Boundaries

- **A. Production Telemetry (Continuous Observability):**
  - **Prometheus Metrics:** Scrapes request throughput, HTTP 5xx error rate, service health, and P95 latency across all services.
  - **Loki Logs:** Streams structured application error events, stack traces, and exception details filtered by service name.
  - **Jaeger Traces:** End-to-end distributed traces capturing request execution waterfalls and isolating failed downstream spans.
- **B. Incident & Failure Signals (Anomaly Triggers):**
  - **Service Failures:** Live microservice outages, error surges, and HTTP 5xx spikes on upstream/downstream routes.
  - **Latency Anomalies:** Degraded transaction durations and response timeout spikes detected across active endpoints.
  - **Controlled Fault Injections:** Dynamic fault scenarios (`http_500_spike`, crash, latency) injected via port 8005 in controlled demo.
- **C. Incident Context & Knowledge Base (RAG):**
  - **Incident Metadata:** Unique incident identifiers (`INC-CD90E2`), affected service targets, timestamp, and status.
  - **Collected Evidence:** Centralized incident evidence repository aggregating linked metrics, logs, traces, and fault metadata.
  - **Operational Runbooks:** Top-k historical postmortems and runbooks retrieved from PostgreSQL using pgvector similarity search.
- **D. Operational Constraints & Policy Boundaries:**
  - **Allowlisted Actions:** Remediations strictly restricted to `RESTART_SERVICE`, `SCALE_SERVICE`, and `ROLLBACK_SERVICE`.
  - **Risk Policy & Safety:** Deterministic parameter validation schemas, risk rules (`LOW`/`MED`/`HIGH`), and approval mandates.
  - **Execution Restrictions:** Simulated-only actuation boundaries; zero direct `kubectl` or cluster mutation permitted.

INGESTION & INVESTIGATION PIPELINE FLOW

Metrics + Logs + Traces + Incident Context ➔ SentryOps Investigation Pipeline



CONCRETE DEMO INPUT EXAMPLE (INC-CD90E2)

- **Controlled Scenario:** Payment service begins returning HTTP 500 errors during controlled `http_500_spike` experiment (`FAULT-1D8A4DA5`, `error_rate: 1.0`).
- **Prometheus Telemetry:** 10/10 order requests fail with HTTP 500. Error rate spikes to 100%; avg latency drops to 1.008s due to fast-failing responses.
- **Loki Error Logs:** Captures structured HTTP 500 Internal Server Error logs and exception stack traces originating from `payment-service` pod.
- **Jaeger Traces (`EV-TRACE-3F77AC`):** Harvests 10 traces across transaction graph, isolating 7 downstream HTTP spans with `error=true` tags.
- **Context & Runbooks:** Correlates active synthetic fault metadata (`EV-CHANGE-9964BD`), pod health (`EV-INFRA-5B1A7A`), and 3 pgvector runbooks (`EVID-KNOW-6750`).

Project Output: What Does SentryOps Produce?

Root Cause Diagnosis, Remediation Proposals, Safety Decisions & Verification Outcomes

• A. Incident Diagnosis & Root Cause Analysis (RCA):

– **Incident Attribution:** Pinpoints affected microservice (`payment-service`), root cause classification, and confidence score.

– **Evidence Linkage:** Grounds diagnosis in verified telemetry (`EV-TRACE-3F77AC`, `EV-CHANGE-9964BD`, `EV-INFRA-5B1A7A`).

– **Anti-Hallucination Gate:** Strictly validates that all cited evidence IDs exist in the incident evidence store before finalizing RCA.

• B. Structured Remediation Proposal:

– **Allowlisted Proposal:** Generates safe remediation action: `SCALE_SERVICE` for `payment-service` (`replicas: 2`).

– **Deterministic Risk Rating:** Classified as `MEDIUM` risk by non-LLM policy engine based on parameter scope.

– **Mandatory Human Gate:** Enforces `requires_approval: true`, automatically halting workflow at `PENDING_APPROVAL`.

• C. Safety Decision & Execution Result:

– **Approval / Rejection Status:** Audit trail records operator decision (`APPROVED` or `REJECTED`) with decision timestamps.

– **Simulated Execution Status:** Execution status: `SUCCESS` applied strictly against in-memory `SIMULATED_INFRA_STATE`.

– **No Live Cluster Mutation:** Explicit boundary: Remediation updates simulated state only; `kubectl get deployment` remains at 1 replica.

• D. Verification & Incident Outcome:

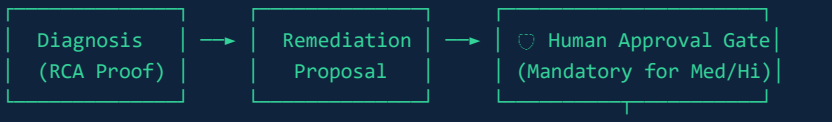
– **Deterministic Verification:** Automated checks confirm simulated replica count (2) and healthy status (`VERIFIED_SUCCESS`).

– **Lifecycle Resolution:** Approved incidents transition to `RESOLVED`; rejected incidents cancel execution and remain `INVESTIGATING`.

– **Audit Postmortem & Evaluation:** Generates 9-section structured postmortem deterministically; passes all 4 evaluation dimensions.

OUTPUT & SAFETY DECISION LIFECYCLE FLOW

Incident Evidence ➔ Diagnosis ➔ Proposal ➔ [Human Approval Gate] ➔ Result



[APPROVED] ⟶ Simulated Exec ⟶ Verification Checks ⟶ RESOLVED

[REJECTED] ⟶ Action Cancelled (null) ⟶ Stays in INVESTIGATING

CONCRETE DEMO OUTPUT & RECOVERY LIFECYCLE

• **Diagnosis & Proposal:** RCA diagnosed payment-service failure. Proposed `SCALE_SERVICE` (replicas: 2), rated `MEDIUM` risk (`PENDING_APPROVAL`).

• **Approval Path (`INC-CD90E2`):** Operator approved (`POST /approve`) ➔ status `APPROVED` ➔ updated simulated state (`SUCCESS`) ➔ verified healthy ➔ status `RESOLVED`.

• **Rejection Path (`INC-28EE86`):** Operator rejected (`POST /reject`) ➔ status `REJECTED` ➔ execution cancelled (`null`) ➔ verification skipped ➔ remains `INVESTIGATING`.

• **Simulated vs Real Actuation:** Remediation executor updated in-memory simulated state only. Live cluster deployment (`payment-service`) remained strictly at 1 replica.

• **Actual Live Recovery Truth:** Live HTTP traffic recovered (10/10 OK, 1.051s latency) strictly following explicit fault cleanup (`POST /cleanup`), NOT from simulated scaling.

Observable Telemetry & Operator Dashboard

Real-Time Telemetry Visibility Across Prometheus, Loki, Jaeger & React 19

- **Grafana Observability Dashboard**:

– **Service Health Status**: Real-time status panel tracking health across all 5 microservices (`order`, `payment`, `notification`, `user`, `api-gateway`).

– **Request Rate**: Per-second throughput across services.

– **Error Rate (5xx)**: Instant visibility into error spikes during active faults; returns deterministically to 0 upon recovery.

– **P95 Latency**: Histogram quantiles tracking delays.

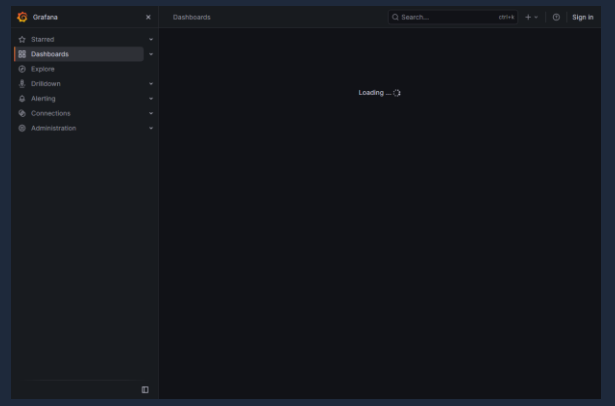
– **Live Service Logs**: Real-time Loki log streaming filtered by `service_name`.

- **Jaeger Distributed Tracing**:

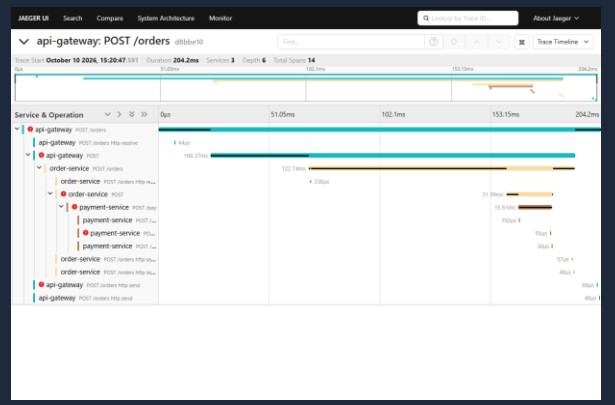
– End-to-end trace waterfalls capturing cross-service HTTP calls and pinpointing error spans.

- **React 19 SRE Incident Dashboard**:

– Interactive incident timeline, evidence inventory, RCA summary, human approval card, and postmortem view.



Grafana: Error Rate (5xx) Spike During Active Fault

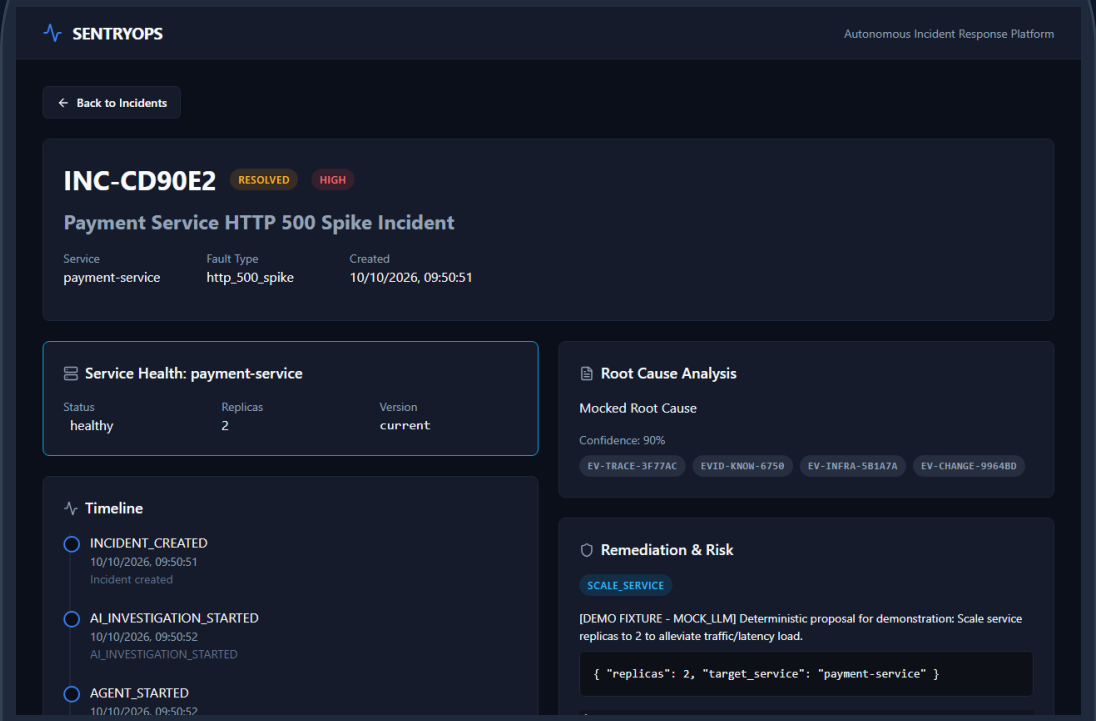


Jaeger: Distributed Trace Waterfall Pinpointing Error Span

Live Demonstration Walkthrough

Validated End-to-End Scenario: http_500_spike on payment-service

- **Incident & Fault Identifiers**:
  – Incident ID: `INC-CD90E2` | Fault Injection ID: `FAULT-1D8A4DA5`.
- **Quantitative HTTP Traffic Verification**:
  – **Baseline**: 10/10 OK (100% success, avg latency: 1.116s).
  – **Fault Active**: 0/10 OK, 10/10 500s (100% failure, avg latency: 1.008s).
  – **Fault Cleanup**: 10/10 OK (100% recovered, avg latency: 1.051s).
- **Harvested Telemetry Evidence**:
  – `EV-TRACE-3F77AC`: 10 Jaeger traces analyzed (7 error spans identified).
  – `EV-CHANGE-9964BD`: Active synthetic fault metadata detected.
  – `EV-INFRA-5B1A7A`: Prometheus node/pod infrastructure health verified.
  – `EVID-KNOW-6750`: 3 relevant historical runbooks retrieved from pgvector.
- **Remediation Proposal & Risk Gate**:
  – Proposal: `SCALE_SERVICE` for `payment-service` (`replicas: 2`).
  – Labeled: `[DEMO FIXTURE - MOCK_LLM]`.
  – Safety Gate: Classified as `MEDIUM` risk -> Halts at `PENDING_APPROVAL`.

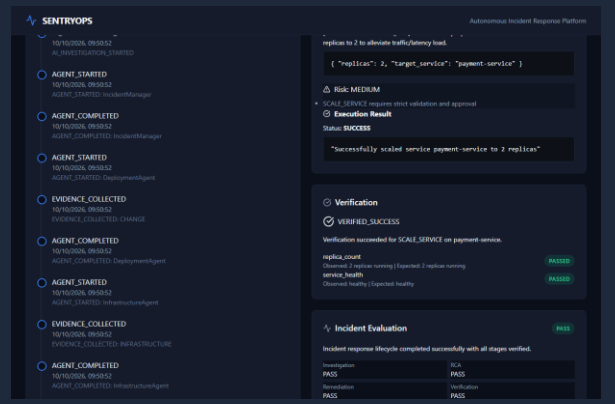


React 19 SRE Dashboard: Remediation Proposal & Risk Assessment

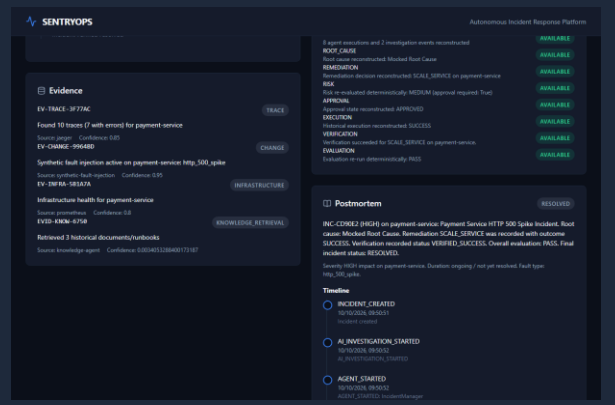
Execution, Verification & Postmortem

Deterministic Lifecycle Completion & Evidence Verification

- **Human Operator Approval Gate**:
  – Operator submitted approval via dashboard (`POST /incidents/{id}/approve`).
- **Simulated Execution Safety Boundary**:
  – Execution status: `SUCCESS` against in-memory `SIMULATED_INFRA_STATE`.
  – Zero Kubernetes cluster mutation: `kubectl get deployment payment-service` remained strictly at 1 replica.
- **Deterministic Post-Remediation Verification**:
  – Check 1 (`replica_count`): 2 replicas verified (Passed).
  – Check 2 (`service_health`): Healthy status verified (Passed).
  – Outcome: Marked `VERIFIED_SUCCESS` -> Incident resolved.
- **Incident Replay & Evaluation**:
  – Evaluation Framework: `PASS` across all 4 dimensions (Investigation, RCA, Remediation, Verification).
  – Replay Consistency: `MATCH` (verified without historical re-execution).
  – Postmortem: 9 structured sections generated deterministically.



Approval Gate & Deterministic Health Verification



Automated 9-Section SRE Incident Postmortem View

Technical Testing & Metric Disclosure

Evidence-Based Verification Across Test Suites & Build Pipelines

- **Kubernetes-Compatible Test Suites**:
  - **152 / 152 PASSED (100% Pass Rate)** in 27.78 seconds.
  - Covers Phase 7 Knowledge Agent, Phase 8 Workflow Demo, Phase 8 Remediation, Phase 9 Verification, Phase 11 Evaluation, Phase 12 Replay/Postmortem, and Phase 13 Kubernetes architecture.
  - **Zero Failures, Zero Skipped**.
- **Frontend Production Build**:
  - `npm run build` completed cleanly in 1.96s with zero errors or warnings.

- **Full Repository Test Suite Disclosure**:
  - **169 Passed, 15 Failed** out of 184 total tests.
  - **Root Cause of 15 Failures**: Exclusively legacy Docker Compose integration tests asserting obsolete host port mappings (`BACKEND_API = 8080` vs `8000`, and direct host ports `8001-8004` which are internal Kubernetes ClusterIPs).
  - **Zero Code Regressions** across all core application logic and agent orchestration workflows.

Security Architecture & Operational Boundaries

Hardened Production Guardrails & Local Prototype Disclosures

• **Hardened Production Constraints**:

– **No Pod Privilege Escalation**: All application pods configure `automountServiceAccountToken: false`.

– **Zero RBAC Abuse**: No Kubernetes cluster-admin or service account permissions granted to application pods.

– **No Real Cluster Actuation**: LLM agents have zero access to shell commands or live Kubernetes APIs.

– **Allowlist Enforcement**: Actions strictly restricted to 3 deterministic operations (`RESTART`, `SCALE`, `ROLLBACK`).

• **Disclosed Local Development Boundaries**:

– **Authentication / Authorization**: FastAPI backend has no token-based authentication middleware (scoped for local prototype demonstration).

– **Default Credentials**: Development manifests configure default database passwords (`sentryops:sentryops`).

– **Insecure Metrics TLS**: Kind Metrics Server configured with `--kubelet-insecure-tls` for local container scraping.

Summary & Future Roadmap

Key Achievements & Next Steps for Production Evolution

- **Key Project Achievements**:
  - Complete, observable, multi-agent incident response lifecycle from alert to postmortem.
  - **100% passing** Kubernetes-compatible test suites (152/152).
  - Deterministic safety gates preventing AI hallucination and unauthorized actuation.
  - Full-stack integration with Prometheus, Loki, Jaeger, Grafana, and React 19 UI.

- **Future Strategic Roadmap**:
  - **Production LLM Integration**: Connecting live models (Claude 3.5 Sonnet / GPT-4o) under the same deterministic safety gates.
  - **Enterprise Security**: Multi-tenant RBAC and API key authentication on incident management endpoints.
  - **GitOps-Driven Actuation**: PR-based remediation proposals via ArgoCD or Flux.
  - **Cloud Observability**: Persistent storage provisioned on AWS EKS or GCP GKE.